

ENGINEERING WALKTHROUGH

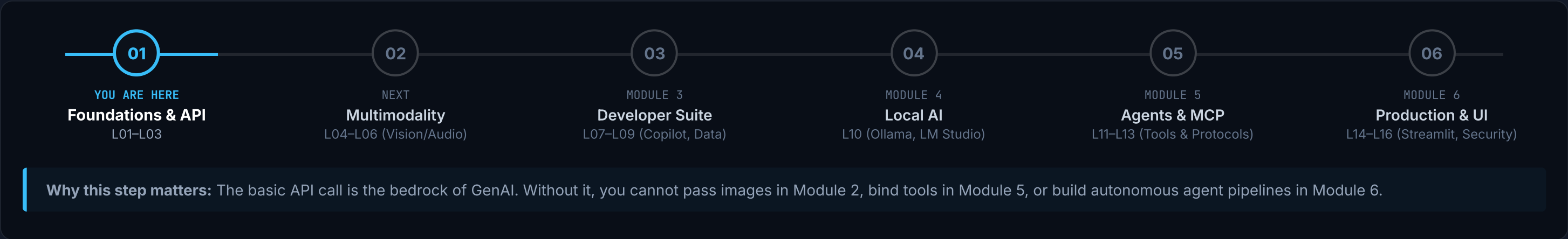
LLM API with Python

Architecture, Client Libraries, and Your First Programmatic Call

Transitioning from interactive browser chats to automated workflows. Exploring client-server network architecture, client SDKs, and enforcing deterministic machine-readable outputs.

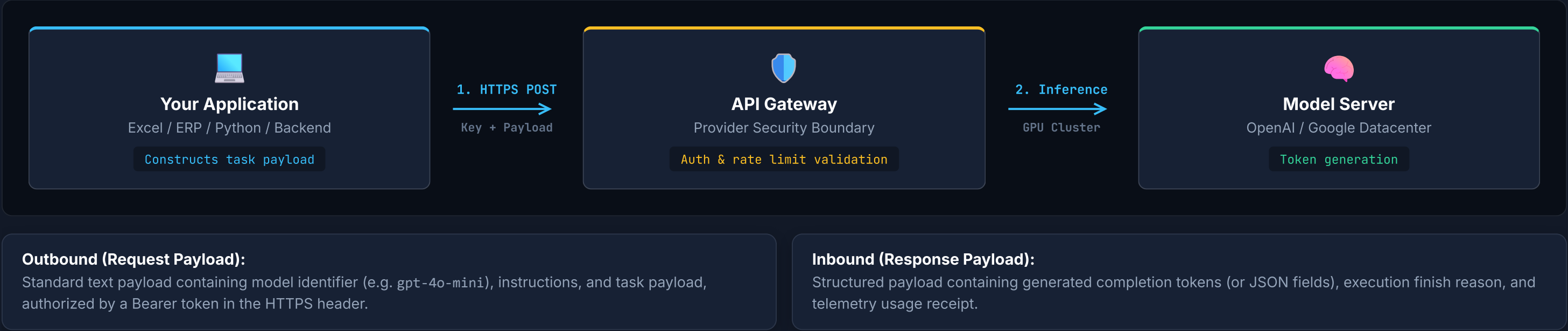
Course Roadmap: Where We Stand

Concluding Module 1: Transitioning from token mechanics to programmatic API integration



How Software Communicates with AI: Architecture Overview

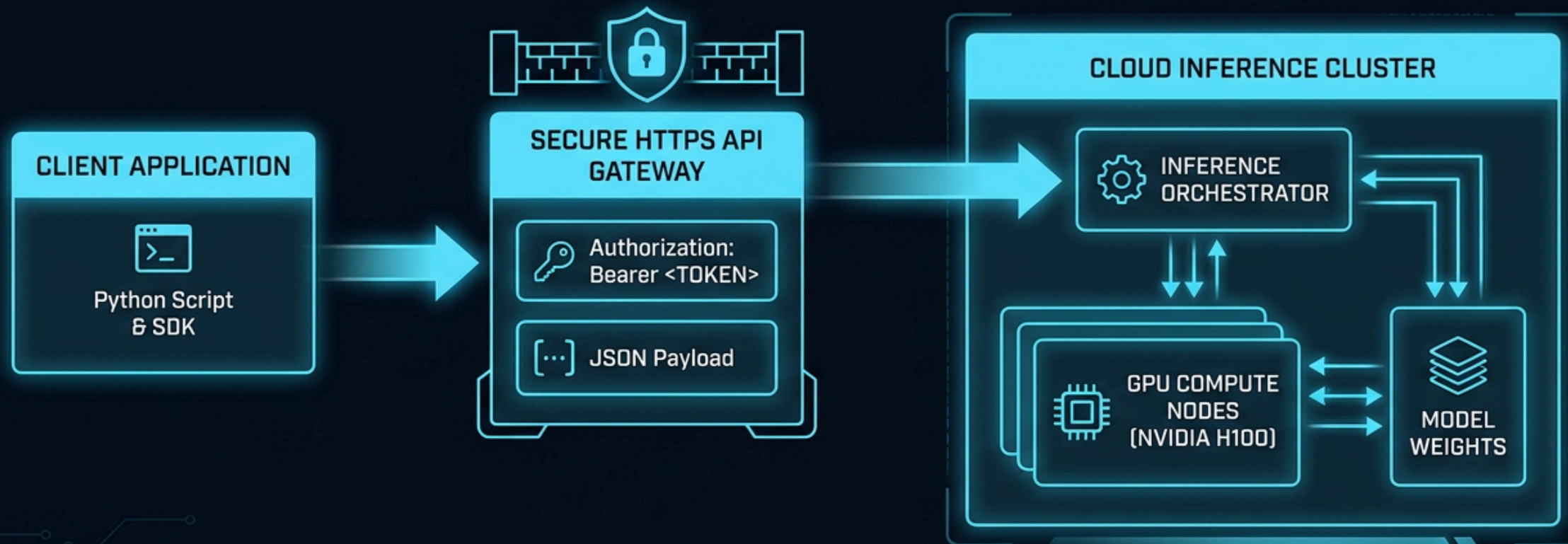
The model does not live inside your local application. It operates as a remote cloud microservice



Anatomy of an HTTP Request: Client, Gateway, and Cluster

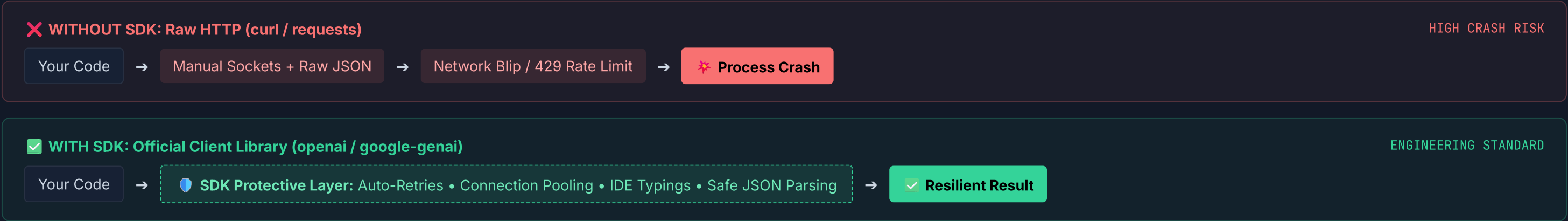
End-to-end packet journey from local script through the REST API boundary to cloud inference accelerators

LLM API INTEGRATION: CLIENT TO CLOUD REQUEST FLOW



Why Use an SDK: The Protective Shield for Enterprise Code

Client libraries handle 90% of low-level networking plumbing and safeguard applications from unhandled crashes



An SDK is not unnecessary complexity; it is pre-built armor. When an API server drops a connection or returns 429, the SDK applies exponential backoff automatically.

First API Call: From Web Playground to Google Colab

Google AI Studio visually configures prompts with a "Get Code" button, while Colab executes scripts with zero key leakage

Google Colab Notebook Cell ▶ Play Button

```
from google import genai
from google.colab import userdata

# 1. Secret shield: fetch from Colab Secrets (🔑 on left panel)
# Bound to your private Google profile: NEVER visible on screen or saved in .ipynb
api_key = userdata.get("GEMINI_API_KEY")
client = genai.Client(api_key=api_key)

# 2. Invoke modern high-efficiency 2026 foundation model
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents="Explain APIs in 2 words."
)

# 3. Print extracted completion directly
print(response.text)
```

- 1. Google AI Studio (Google Labs)**
The playground at `aistudio.google.com`: experiment with prompts in UI, then click "**Get Code**". Writing Python from scratch is completely optional!
- 2. Security: Colab Secrets 🔑**
Secrets stay in the system vault `userdata.get()`. No intrusive getpass popups and zero risk of leaking quota balances during live screenshares.
- 3. Gemini 3.5 Flash Model**
Current 2026 high-speed reasoning standard (2M+ token window, native multimodal video/doc ingestion, 15 RPM free tier).

What the Server Returns: Dissecting the Digital Receipt

Alongside text, the model returns a transaction passport: finish reason and exact token accounting

OUTPUT PAYLOAD

``response.text``

Clean completion text. The SDK automatically stitches multi-candidate text parts and cleans up transport headers.

FINISH STATUS

``finish_reason``

STOP confirms regular thought completion. MAX_TOKENS signals ceiling reached before completion.

TOKEN TELEMETRY

``usage_metadata``

Audits exact consumption: `prompt_token_count` (input) and `candidates_token_count` (output) for quota oversight.

```
# Verification of receipt in Google Colab:
m = response.usage_metadata
print(f"Tokens: Prompt={m.prompt_token_count}, Output={m.candidates_token_count}, Total={m.total_token_count}")
print(f"Finish Reason: {response.candidates[0].finish_reason}")
```

API Key Security: Protecting Credentials and Budgets

An API key is an active credit line with direct access to your corporate or personal billing account



Public GitHub Leak in Seconds

Automated bots scan public commits 24/7. Hardcoded keys in scripts get hijacked in seconds, incurring runaway cloud charges.



Standard: .env & .gitignore

Keys are kept locally in a .env file, which is strictly tracked inside .gitignore so Git never commits it to version control.



Colab Secrets (🔑 Tab)

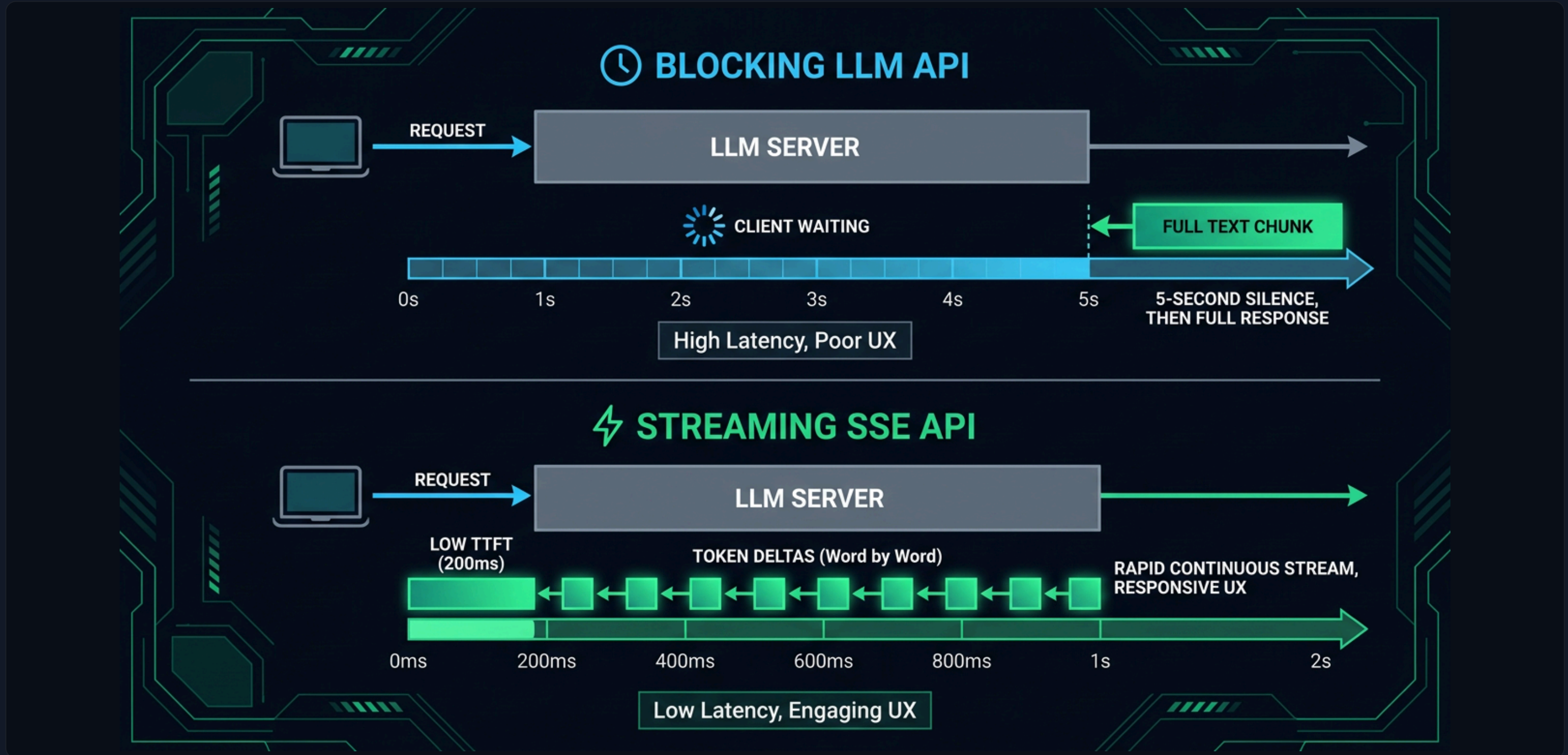
In Colab's left sidebar, click the key icon 🔑 → save GEMINI_API_KEY. The token binds to your Google account, not the file! Sharing the notebook never exposes your key.

```
# 1. Local development: .gitignore shields .env from Git leakage:
.env # Prevents catastrophic git push of credentials to public repos

# 2. Google Colab: fetch from Secrets store (token never displays on screen or video):
from google.colab import userdata
api_key = userdata.get("GEMINI_API_KEY") # Injected securely into memory without showing string
```

Streaming: Server-Sent Events Physics and Latency

Timeline comparison: 5 seconds of blank waiting versus immediate first-token delivery (TTFT)



Streaming in Code: Dissecting Real-Time SSE Generation

The `generate_content_stream` method delivers token deltas as they compute, eliminating perceived lag

```
# Enable real-time token streaming (Server-Sent Events):
response = client.models.generate_content_stream(
    model="gemini-3.5-flash",
    contents="Explain streaming in 2 sentences."
)

# Output token deltas to console without buffering:
for chunk in response:
    print(chunk.text, end="", flush=True)
```

What is a `chunk`?

A lightweight socket payload containing an incremental delta—typically 1 or 2 freshly generated tokens rather than the full document.

Why `flush=True`?

Forces the system I/O buffer to flush immediately to the terminal/UI, producing the fluid real-time typewriter experience.

When NOT to Stream?

Avoid streaming for background automation, batch ETL pipelines, or strict JSON extraction. Use standard blocking calls for atomic outputs.

From Chat to Code: Translating XML & One-Shot into API Calls

Curriculum skills become code parameters: system instructions, XML demarcation, and deterministic greedy decoding

```
from google.genai import types

# 1. System instructions decoupled from user input (Anti-Injection)
config = types.GenerateContentConfig(
    system_instruction="You are a strict compliance auditor. Extract facts without pleasantries.",
    temperature=0.0 # 0.0 = maximal determinism
)

# 2. XML structure and One-Shot exemplar inside f-string:
prompt = f"""
<context>Inbound customer billing dispute</context>
<rules>Extract disputed amount and responsible department</rules>
<example>
    <input>Erroneously billed $500 twice for annual license</input>
    <output>department: billing | amount: $500</output>
</example>
<document>{raw_text}</document>
"""

response = client.models.generate_content(
    model="gemini-3.5-flash", contents=prompt, config=config
)
```

1. `system\_instruction` Parameter

Architecturally isolates system authority from untrusted input text. Prevents user prompt injection attempts from overriding enterprise rules.

2. XML Tags inside F-Strings

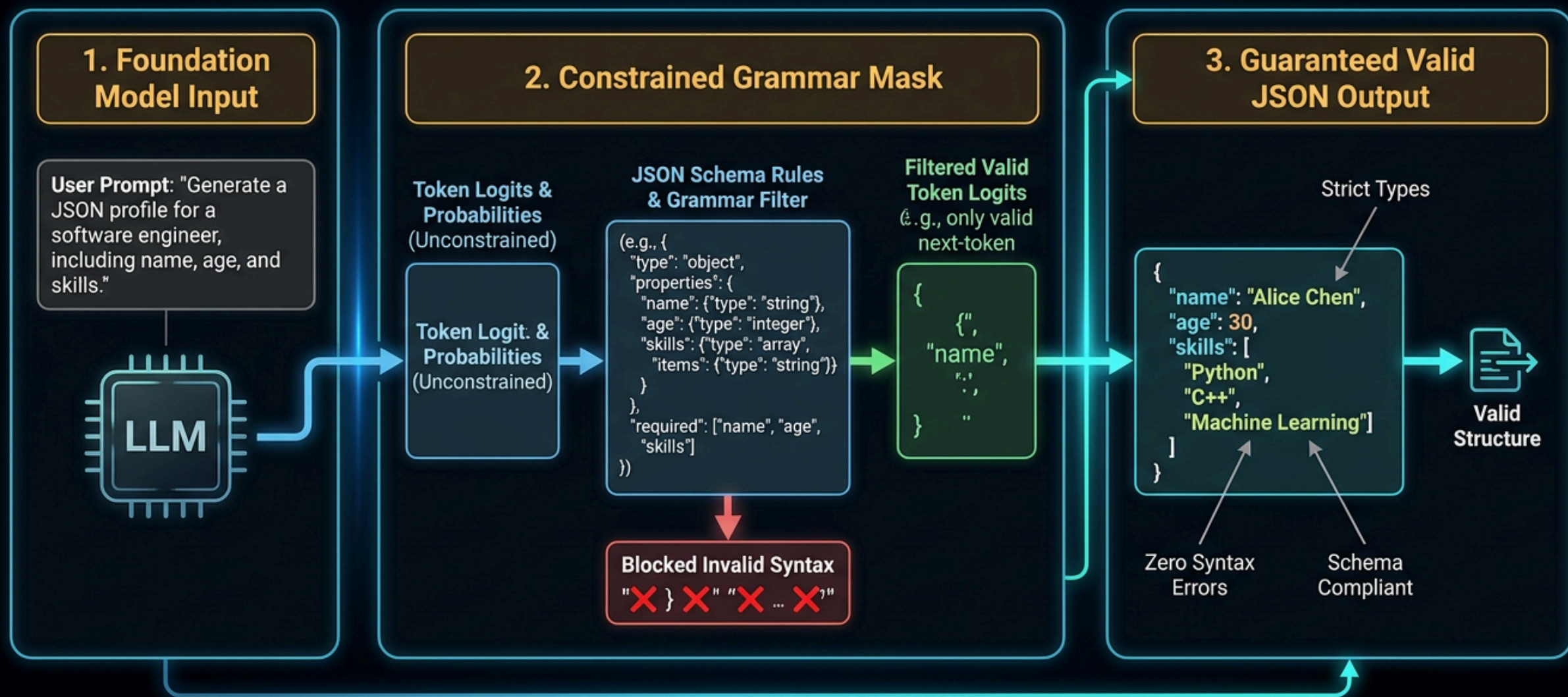
Tags like <context>, <rules>, and <document> establish unambiguous semantic boundaries for foundation models.

3. One-Shot Exemplar + `temp=0.0`

Supplying a single concrete Input → Output exemplar with greedy decoding eliminates conversational fluff and guarantees consistent schemas across runs.

Constrained Decoding: A grammatical mask zeros out the probabilities of invalid tokens during autoregression

Structured Outputs: Constrained Decoding Mechanics



Practical Implementation: Parsing Inbound Emails into JSON

Using `response\_mime\_type="application/json"` and temperature=0.0 for deterministic schema contracts

```
import json
from google.genai import types

# Constrained mode: model is physically barred from emitting non-JSON tokens
config = types.GenerateContentConfig(
    response_mime_type="application/json",
    temperature=0.0 # Zero randomness, strict greedy decoding
)

prompt = """
Extract ticket fields into JSON:
{
  "category": "billing" | "technical" | "general",
  "urgent": true | false,
  "amount_usd": float
}
Ticket: URGENT! Card was charged twice for $450 subscription, refund immediately!
"""

response = client.models.generate_content(
    model="gemini-3.5-flash", contents=prompt, config=config
)

# Parse guaranteed valid JSON directly into Python dictionary:
data = json.loads(response.text)
print(data["category"]) # → "billing"
print(data["amount_usd"]) # → 450.0
print(data["urgent"]) # → True
```

What Happened?

An emotional unstructured customer email was transformed into structured data fields in 1 second flat.

Why `temperature=0.0`?

Creative sampling risks hallucinating numbers. Zero temperature enforces greedy decoding and maximum determinism.

Downstream Routing

The amount_usd field directly updates ledger records, while urgent complaints escalate to billing staff automatically.



Midpoint Milestone: Coffee Break

Take a 5-minute breather. In part two, we explore network error handling, the ecosystem landscape, and run our interactive notebook in Google Colab.

05 : 00

START 5 MIN

RESET

What Can Go Wrong: 4 Common Network Failures

Connections drop, cloud providers face outages, and quotas deplete. How robust software handles failure

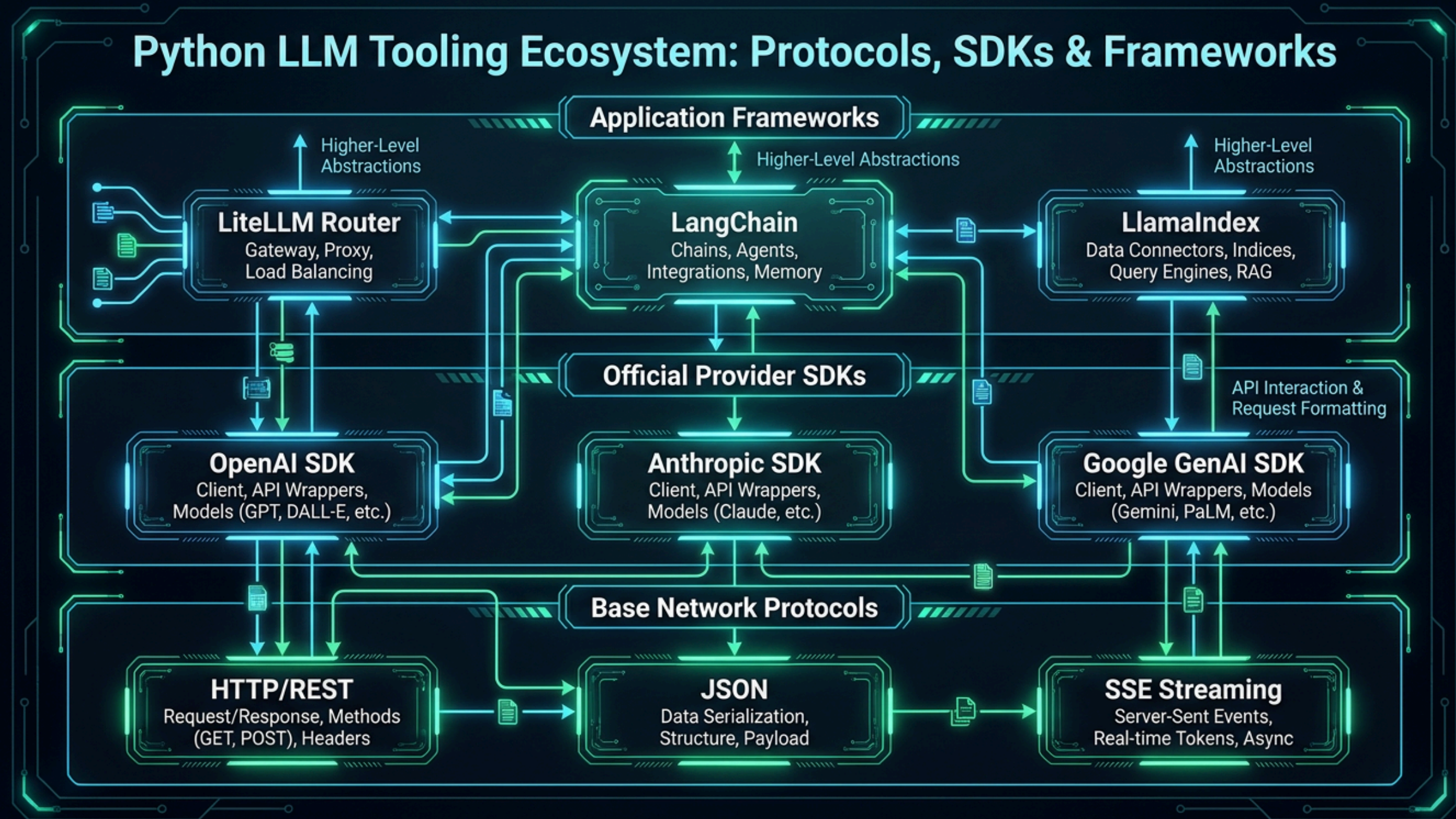
401 UNAUTHORIZED Invalid Key Key copied with trailing space, revoked, or missing in environment. Action: verify token value and access rights.	429 RATE LIMIT Quota Reached Exceeded requests-per-minute threshold or depleted credit tier. Action: backoff delay or tier upgrade.	500 / 503 ERROR Provider Outage Temporary overload in OpenAI or Google datacenters. Action: automated retry or multi-provider fallback.	TIMEOUT Socket Stall Packets stalled mid-transit. Without explicit timeout=30.0, applications hang indefinitely.
---	--	--	---



Built-in Retries: Official SDKs bundle retry logic for 429 and 500 errors out of the box, saving you from writing custom retry loops.

Tooling Landscape: Who's Who in the Python AI Stack

From low-level HTTP protocols to official vendor SDKs and high-level agent orchestration frameworks



Provider Portability: OpenAI vs Claude vs Gemini

Different method names, identical mental models. Mastering one SDK equips you for every provider

OPENAI SDK (2026)

Function Calling Standard

```
client.chat.completions.create(  
  model="gpt-4o", # or o3-mini  
  response_format={...}  
)
```

Industry reference for Structured Outputs and Function Calling. Supported natively by local inference servers (vLLM, Ollama).

ANTHROPIC SDK (2026)

Deep Reasoning & Code

```
client.messages.create(  
  model="claude-3-7-sonnet",  
  thinking={"type": "enabled"}  
)
```

Flagship for architectural reasoning and code generation. Supports hybrid thinking mode with adjustable reasoning budgets.

GOOGLE GENAI SDK (2026)

2M+ Window, Video & Free Tier

```
client.models.generate_content(  
  model="gemini-3.5-flash",  
  contents=[...]  
)
```

Fastest enterprise tier, native multimodal audio/video understanding, 2M+ context window, and 15 RPM free tier without foreign credit cards.

💡 Open-weight models like **DeepSeek-V3 / R1** and unified libraries like **LiteLLM** allow swapping providers with a single line of code.

First AI Script Checklist: 5 Core Rules

The pre-flight checklist before handing code to users or running scripts in production

1. Secret Hygiene

Keys stored strictly in `.env` or environment variables. The `.env` file is explicitly tracked in `.gitignore`.

MANDATORY

2. Explicit Timeouts

Always supply `timeout=30.0` when initializing client instances to prevent deadlocked threads from network hangs.

RESILIENCE

3. Calibrated Temperature

`temperature=0.0` for JSON schemas, numbers, and classification tasks. `0.7+` only for creative prose generation.

ACCURACY

4. Schema Enforcement

Never rely on prompt pleas like "return JSON". Enforce structured contracts via the API's native `response_format` parameter.

CONTRACT

5. Telemetry & Audit Logging

Always persist `response.id` and `usage.total_tokens` to maintain financial traceability and performance audits.

GOVERNANCE

Homework: Run in Colab & YouTube WOW-Case

Two tracks to choose from: standard document triage into JSON, or an automated video analysis pipeline

TRACK A • FOUNDATIONAL

Workplace Document Triage to JSON

- 1. Take a real paragraph from your routine work (email, bug ticket, invoice).
- 2. Run the provided notebook L03_Google_Colab_Quickstart.ipynb.
- 3. Extract structured result.json with mandatory fields (category, amount, priority).

Focus: Constrained Decoding & Pydantic

TRACK B • WOW EFFECT (CREATOR TOOLS PATTERN)

YouTube & Meeting Video Analyzer

- 1. Supply any YouTube URL (webinar, tech talk, or lecture) in Colab.
- 2. Script extracts transcript in 1 sec, formats it via Lesson 02 XML template, and dispatches to gemini-3.5-flash.
- 3. Model generates an **Executive Briefing Memo**: TL;DR bullets, timestamped milestones, and actionable tasks.

Focus: Save 2 hours of video watching

Q&A: Core Takeaways & Next Session

Open floor discussion and preview of Module 2 — advancing into images and multimodal models

3 CORE ARCHITECTURAL TAKEAWAYS

1. **LLM is a remote cloud server.** No models are downloaded locally; you exchange tokens over standard HTTPS.

2. **Streaming is for humans, blocking is for machines.** Stream when user eyes are on screen; block for automated pipelines.

3. **Structured JSON protects your database.** Fix schemas with `response_format` and keep temperature at 0.0.

NEXT SESSION • LESSON 04

Image Generation and Editing

Advancing into **Module 2: Multimodality**.

Exploring diffusion architectures: **DALL-E 3**, **Midjourney**, **Stable Diffusion**, and **Google Imagen**.
Generating technical diagrams and manipulating visual assets programmatically.

 Microphones are open — questions welcome in chat and on mic!